



Lab 03: Graph Coverage, Part 2

Objective: This lab extends the CFG and coverage-criteria skills from Lab 02 to more realistic code: methods that combine nested loops, break and continue, exception handling, and multiple return statements. Students will build two CFGs, work out how each construct changes the graph's node and edge structure, and design test paths that achieve Node Coverage and Edge Coverage on both.

Introduction

Lab 02 built CFGs from single loops, if/else-if chains, and a switch statement. Real code rarely stays that simple: loops nest inside other loops, a `break` or `continue` can redirect control away from the "next line," a method can return from several different places, and a statement that looks harmless on the page can fail at runtime and jump to a `catch` block instead of continuing normally. This lab works through two methods that combine several of these features at once, so you can practice reading a CFG where the "obvious" next node is not always the right one.

New node/edge patterns in this lab:

- **Multiple final nodes:** a method with several `return` statements has one final node per return, so Nf can contain more than one node.
- **break:** creates an edge straight from the point of the break to the first node after the enclosing loop, skipping the loop's own condition-check node.
- **continue:** creates an edge back to the loop's condition-check node (in a `while` loop) or to wherever the loop's own update step happens next (in a `for` loop); it does not skip the rest of the loop the way `break` does, it only skips the rest of the current iteration's body.
- **try/catch:** a statement that can throw an exception gets a second outgoing edge, straight to the first node of the matching `catch` block, in addition to its normal

edge to whatever follows it when no exception occurs. A method can also throw explicitly with a `throw` statement; that is its own node, and its edge goes straight to the matching catch block, the same way a normal statement's edge goes to whatever follows it. Different exception types can have different catch blocks, so the statement that might fail can have more than two outgoing edges in total.

- **for loops:** a `for` loop's initialization and its update step (the parts written inside the parentheses) are not shown as separate lines of code, but they still need their own nodes in the graph, just like the lecture's "implicitly initializes loop" and "implicitly increments loop" examples. A `continue` inside a `for` loop goes to that update node, not straight to the condition, because Java always runs the update before re-checking the condition.
- **self-loops:** if a loop's condition check is also its entire body, the true-branch edge does not lead to some other node, it leads straight back to the same node. A node with an edge to itself needs extra care for criteria stronger than Edge Coverage.
- **infeasible test requirements:** not every node or path a criterion asks for can actually be executed. Dead code and redundant checks can make a test requirement impossible to satisfy no matter what input you choose, even though the graph itself looks perfectly normal.

Lab Activity

This is an individual activity. Complete each section on your own and type your answers directly into the boxes below.

Section 1: Fundamentals of Modeling Loops, Exceptions, and Returns

Prepare a concise document covering the following.

1. When a statement inside a try block might throw an exception, why does modeling it in a CFG typically require two outgoing edges from that statement's node instead of one?

2. Explain the difference between how break and continue change the normal flow of edges in a loop's CFG. Which one can create an edge that skips the loop's condition-

check node entirely?

3. Why can a method with multiple return statements have more than one final node in Nf? Does this break any rule about what a CFG is allowed to look like?

Section 2: A Nested Loop with break, continue, and Multiple Returns

The Java method below searches a 2D array (an array of rows) for two side-by-side elements in the same row that add up to `target`. Rows that are missing or too short are skipped. Unlike Lab 02, the code is not pre-numbered: identifying the basic blocks (nodes) is part of the task.

```
public static int[] findPairSum(int[][] matrix, int target) {  
    if (matrix == null) {  
        return null;  
    }  
  
    for (int row = 0; row < matrix.length; row++) {  
  
        if (matrix[row] == null || matrix[row].length < 2) {  
            continue;  
        }  
  
        boolean found = false;  
  
        for (int col = 0; col < matrix[row].length - 1; col++) {  
  
            if (matrix[row][col] + matrix[row][col + 1] == target) {  
                found = true;  
                break;  
            }  
        }  
    }  
}
```

```
        if (found) {  
            return new int[] {row};  
        }  
    }  
  
    return null;  
}
```

Reading the graph: The null-matrix guard at the top has its own exit point, separate from the rest of the method. Both for loops need their initialization and their update step each modeled as their own node, exactly like the lecture's "implicitly initializes loop" and "implicitly increments loop" examples, even though `row = 0` and `row++` are not written as separate lines of code. The outer loop skips a missing or too-short row with `continue`; that `continue`'s edge goes to the outer loop's update node (not straight to the condition), and the loop's normal end-of-body fall-through goes there too, since Java always performs the update before re-checking the condition either way. Finding a match inside the inner loop uses `break`, which jumps straight past the inner loop's condition check and its update node entirely, that is different from the inner loop simply running out of columns, which reaches the same following point but through the condition check's false branch instead, having gone through the update node on every completed iteration. This method has three separate exit points in total: the null-matrix guard, the branch where a match was found, and the point reached once every row has been searched with no match.

Task A: Write down the graph

Break this method into basic blocks and write down the sets N (nodes), N_0 (initial nodes), N_f (final nodes), and E (edges) for this CFG. There is no single correct numbering, choose your own and stay consistent. Pay particular attention to the `break` statement: where does its edge go, and is that the same node the inner loop's own condition reaches when it simply runs out of columns?

Optional but encouraged: draw this CFG in draw.io (or any diagramming tool), export it as a PNG or JPG, and upload it here so you can check your answer visually.

Task B: 100% Node Coverage

Using the graph you wrote for Task A, write a set of test cases (each one a `matrix` and a `target`) that together achieve 100% Node Coverage, that is, every node in your graph is visited by at least one test case. You will need matrices that cover a null or too-short row, a row with no matching pair, and a row with a matching pair, among other cases. State which node(s) each test case visits that the earlier ones did not.

Task C: 100% Edge Coverage

Using the same graph, write a set of test cases that together achieve 100% Edge Coverage, that is, every edge in your graph is toured by at least one test case. Pay attention to the statement that checks whether the pair sum matches the target: it has two outgoing edges, and your test cases need to tour both.

Section 3: A Loop with `try/catch`, `break`, `continue`, and Multiple Returns

The Java method below adds up a list of number strings, stopping early if it sees the sentinel value `"STOP"` , skipping any string that fails to parse as an integer, and rejecting negative values outright. As in Section 2, the code is not pre-numbered.

```
public static int parseAndSum(String[] tokens) {
    if (tokens == null || tokens.length == 0) {
        return 0;
    }

    int sum = 0;
    int index = 0;

    while (index < tokens.length) {

        if (tokens[index].equals("STOP")) {
            break;
        }
    }
```

```

    try {
        int value = Integer.parseInt(tokens[index]);
        if (value < 0) {
            throw new IllegalArgumentException("negative value");
        }
        sum += value;
    } catch (NumberFormatException e) {
        index++;
        continue;
    } catch (IllegalArgumentException e) {
        return -1;
    }

    index++;
}

return sum;
}

```

Reading the graph: The STOP check's `break` jumps straight past the whole loop, skipping the rest of that iteration and every future check of the loop's condition. The statement that parses text to an int is risky: unlike every branch you modeled in Section 2, its second outgoing edge does not come from an if or switch condition written in the source, it comes from the possibility that the statement itself fails at runtime, a normal edge when parsing succeeds, and an exceptional edge to the matching catch block when it throws. The negative-value check adds a different kind of exceptional edge: `throw` is a statement you write yourself, and it is a node like any other, except its one outgoing edge always goes straight to whichever catch block matches its exception type, the same way a normal statement's edge goes to whatever follows it. Because there are two different exception types here, there are two different catch blocks, and they behave differently: the `NumberFormatException` catch increments `index` and continues the loop, while the `IllegalArgumentException` catch returns immediately. The catch block that continues jumps directly back to the loop's condition check; because this is a while loop, not a for loop, there is no separate update node for continue to pass through the way there was in Section 2, which is why that catch block has to increment `index` itself first. This method now has three separate exit points: the empty-input guard, the point reached once the loop ends normally, and the early return triggered by a negative value.

Task D: Write down the graph

Break this method into basic blocks and write down the sets N (nodes), N_0 (initial nodes), N_f (final nodes), and E (edges) for this CFG. Pay particular attention to the statement that parses the string into an int, and separately to the statement that throws for a negative value: how many edges leave each one, and what real-world event does each edge represent?

Optional but encouraged: draw this CFG in draw.io (or any diagramming tool), export it as a PNG or JPG, and upload it here so you can check your answer visually.

Task E: 100% Node Coverage

Using the graph you wrote for Task D, write a set of test cases (each one a `tokens` array) that together achieve 100% Node Coverage. State which node(s) each test case visits that the earlier ones did not.

Task F: 100% Edge Coverage

Using the same graph, write a set of test cases that together achieve 100% Edge Coverage. Make sure your test cases include: the normal edge out of the parsing statement, its exceptional edge to the `NumberFormatException` catch block, and the throw statement's edge to the `IllegalArgumentException` catch block.

Task G: Comparing continue Across Loop Types

In Section 2 (a for loop), the continue statement's edge goes to the loop's own update node (the one holding `row++`), which then flows into the condition node, and the normal end-of-body fall-through goes through that same update node too. In Section 3 (a while loop), the continue statement's edge goes straight to the condition node, with no update node in between, because a while loop has no built-in update step at all, which is why the catch block that continues has to write

`index++` itself first. What would happen to the CFG, and to the method's behavior, if that catch block in Section 3 called `continue` without first doing `index++` ?

Task H: Edge-Pair Coverage and Self-Loops

Edge-Pair Coverage (EPC) is stronger than Edge Coverage: its test requirements are every reachable path of length up to 2, not just length up to 1. Most of the time that just means "every pair of consecutive edges." A self-loop, a node with an edge back to itself, is the case where that stops being obvious. The method below drains a queue by repeatedly polling it; polling is the only thing that happens each iteration, so the loop's condition check and its entire body are the same single statement.

```
public static void drainQueue(Queue<Integer> queue) {
    while (queue.poll() != null) {
        // polling is the only work done each time
    }
}
```

Reading the graph: This method has only two nodes: the `poll() != null` check, and the implicit end of the method once it returns false. Because the check is the entire loop body, a true result does not lead to some other node before looping back, it leads straight back to itself. That is a self-loop: one node with an edge to itself, plus a second edge out to the exit. Edge Coverage only needs one call with an empty queue (tours the exit edge) and one call with at least one element (tours the self-loop edge at least once). Edge-Pair Coverage needs more: it requires every reachable length-2 path, and a self-loop can be immediately followed by itself, so touring the self-loop edge only once is not enough.

Write down the graph and the EPC test requirements

Write down N , N_0 , N_f , and E for this method (it should only need two nodes). Then list the length-2 paths Edge-Pair Coverage requires. You will find one of them needs the self-loop to be toured twice in a row, work out what that means for the size of the queue in that test case, and give a set of test cases (queue contents) that achieves 100% Edge-Pair Coverage.

Task I: Infeasible Test Requirements

Not every test requirement a criterion generates can actually be satisfied. A node can be unreachable (dead code), or a path can require two conditions that contradict each other, such as needing a variable to be both greater than and less than zero at the same point. The method below has a redundant check that creates exactly this kind of problem.

```
public static String classifyAge(int age) {  
    if (age < 0) {  
        return "invalid";  
    }  
    if (age < 0) {  
        return "unreachable";  
    }  
    if (age < 18) {  
        return "minor";  
    }  
    return "adult";  
}
```

Find the infeasible node

Write down N, N0, Nf, and E for this method. One branch's true-edge leads to a node that can never actually be executed by any input, even though the graph itself looks completely normal. Which node is it, and what is the logical contradiction that makes it infeasible? Then explain what this means for 100% Node Coverage on this method: is it achievable, and if not, what is the best a test set can do (tying back to what "syntactically reachable" means in the definition of Node Coverage)?

Discussion

Edge Coverage on Section 3's graph requires touring the exceptional edge out of the parsing statement, the path where `Integer.parseInt` throws. In real testing, exception paths are often under-tested because it is easy to forget to construct inputs that trigger them. How does treating the exception as a normal CFG edge change the way you approach test design?

Compare Section 2's break statement (inside the inner loop) and Section 3's break statement (on seeing the STOP sentinel). Both skip the remaining loop body and jump straight past the loop, but do they always land on the same kind of node? What determines which node a break's edge points to?



FACULTY OF COMPUTING